

Übungsblatt 00

Prof. Dr. Jens von Pilgrim

In diesem Aufgabenblatt wiederholen wir asynchrone Programmierung mit TypeScript/JavaScript und lernen, wie man eine Lösung in WE2 abgibt. Dieses Aufgabenblatt ist noch kein Teilleistungsnachweis im Sinne der Prüfungsordnung.



Dieses Aufgabenblatt ist sehr lang, da alle Schritte ausführlichst erklärt werden. Die Aufgaben selbst sind im Grunde sehr klein und einfach. In späteren Blättern werden, bis auf ein paar Ausnahmen, die Tools nicht mehr erklärt.

Inhaltsverzeichnis

1. Einleitung	2
2. Entwicklungstools	4
2.1. Git	4
2.2. Node.js	5
2.3. Visual Studio Code	6
2.4. Installieren der Jest-Extension	7
2.4.1. Konfiguration der Jest-Extension	7
3. Vorbereitung	8
3.1. Forken	8
3.2. Klonen	9
3.3. VS Code starten und Module installieren	9
4. Zeilen zählen	10
5. Zeilen einer Datei zählen	12
5.1. Synchrone Variante	13
5.2. Variante mit Callback	13
5.3. Variante mit Promise	13
5.4. Variante mit async/await	14
6. Zählen mit Filter	14
7. Einreichung der Lösung	16
8. Abnahme und Bewertung	18
Abkürzungsverzeichnis	19
Literatur	19

1. Einleitung

Lernziele

Sie können asynchrone TypeScript/JavaScript-Programme schreiben,
indem Sie die Tools anwenden, in JavaScript Funktionen als Argumente in Form von Callbacks verwenden, die Promise-API verstehen und `async` / `await` einsetzen und das alles auch noch testen können,
damit Sie mit TypeScript später den Backend- und Frontend-Code erstellen können.

Daneben lernen Sie mit diesem Arbeitsblatt, was Sie generell als Lösung einreichen müssen. Wenn später aus formalen Gründen Ihre Lösung nicht korrigiert werden konnte oder Teile fehlen, wird die Aufgabe mit 0 Punkten bewertet!

Bearbeitungshinweise

Web-Engineering 2 (WE2) setzt solides Programmierwissen in TypeScript und JavaScript voraus. Wer TypeScript noch nicht kennt: In 90 % der Fälle sieht der Code wie JavaScript aus, in wenigen Fällen (vor allem bei formalen Parametern) müssen statische Typannotationen explizit angegeben werden.

JavaScript und TypeScript sind bzw. waren Inhalt von Web-Engineering 1 (WE1). **Wenn Sie WE1 noch nicht belegt oder bestanden haben, wird von einer Teilnahme an WE2 dringend abgeraten.** Falls Sie WE1 erfolgreich absolviert haben, aber Ihre JavaScript-Kenntnisse nicht gut sind, dann müssen Sie diese selbstständig nachholen – am besten sprechen Sie dann mit dem Dozenten. **Wir werden der Einfachheit halber im Folgenden (in diesem Blatt und im gesamten Modul) nur von JavaScript sprechen, auch wenn wir hier ausschließlich TypeScript verwenden werden.**

WE1 ist Voraussetzung!

In diesem Blatt geht es um Programmierung mit TypeScript und JavaScript. Dieses Blatt fließt nicht in die Bewertung ein, sondern ist eher ein Test für Sie, wie gut Ihr JavaScript-Wissen ist. **Wenn Sie mit diesem Aufgabenblatt große Probleme haben sollten, wenden Sie sich bitte an den Dozenten.**

Quasi als Checkliste für Sie hier die wichtigsten Themen, die Sie bereits kennen sollten:

- **Funktionale Programmierung** mit JavaScript/TypeScript: Funktionsreferenzen als Werte für Variablen, anonyme Funktionen (Lambdas) mittels Function Expressions oder Arrow-Function-Definitionen
- **Asynchrone Programmierung** mit JavaScript/TypeScript: Grundlagen (Single Thread bzw. Single Call Stack, Event Loop); die drei Techniken Callbacks, Promise-API und Async/Await

- **Objektorientierte Programmierung** mit JavaScript/TypeScript: Objekterzeugung durch Konstruktor-Funktionen, Klassen
- **Typannotationen in TypeScript**: Colon-Syntax, spezielle Konstrukte (Union Type, Intersection Type, Type Alias), Überladen von Funktionen
- **Node.js**: Node.js als Ausführungsumgebung, Node.js-Application Programming Interface (API) (Filesystem), Node Package Manager (NPM),
📄 [package.json](#), 📁 [node_modules](#)
- **Test-Driven Development (TDD)** mit **Jest**: Schreiben von Tests mit Jest, Code-Coverage
- **Git und GitLab**: Der Einsatz von Git ist verpflichtend (siehe auch Abschnitt 8)! Forken von Projekten auf GitLab, GitLab-CI (die Konfigurationsdatei wird mitgeliefert)

Konventionen in den Übungsblättern

In den Übungsblättern werden diverse Symbole verwendet, die Ihnen bei der Bearbeitung helfen sollen. Folgende Symbole werden verwendet:

⚠️ | Hinweise, die wichtige Informationen zur Bearbeitung enthalten und die auf jeden Fall ernst genommen werden sollten.

➡️ | Besonders wichtige Angaben, was konkret bei Aufgaben verlangt wird und was, insbesondere hinsichtlich der Korrektur, unbedingt einzuhalten ist. Falls Sie gegen diese Anweisungen verstoßen, droht massiver Punktabzug oder sogar 0 Punkte für die Aufgabe. Oft hängt dies auch mit der automatischen Korrektur zusammen.

💡 | Ein Hinweis mit Tipps für die Umsetzung. Diese Hinweise sind nicht verbindlich, sparen aber unnötige Suche mit Google und sind außerdem auf die Aufgaben abgestimmt.

🌐 | Hintergrundinformationen, die für die Bearbeitung der Aufgabe nicht entscheidend sind, aber das Verständnis erleichtern sollen.

🤖 | Sie werden im Text ab und an Hinweise zum Umgang mit generativer Artificial Intelligence (genAI) finden. Generell ist es sehr schwer, konkrete Hinweise zu geben, da sich genAIs ständig weiterentwickeln und die Nutzung stark vom Kontext abhängt.

🕒 | **Arbeitsaufwand**: Das gesamte Blatt sowie (Teil-) Aufgaben enthalten Angaben, wie lange Sie ungefähr für die Bearbeitung benötigen. Diese Angaben gehen immer davon aus, dass Sie den Stoff bereits einigermaßen gelernt haben und Sie das benötigte Vorwissen haben.

🔒 | Sie müssen Zwischenstände über das Versionsverwaltungssystem Git protokollieren, dies wird bei der Korrektur auch geprüft.

Literaturhinweise

Zum Lernen und Nachschlagen werden die folgenden beiden Bücher dringend empfohlen. Beide sind online über die BHT-Bibliothek verfügbar (einzelne Teile können als PDF heruntergeladen werden):

- David Flanagan: „JavaScript – Das Handbuch für die Praxis“^[6]
- Boris Cherny : „Programmieren in TypeScript“^[4]

Leider ist die ECMAScript-Spezifikation [5] nicht als Lektüre zu empfehlen (im Gegensatz zur sehr guten Java-Spezifikation). Dafür gibt es eine sehr gute Online-Quelle, die auch ständig aktualisiert wird. Unter

<https://developer.mozilla.org>

findet man sehr viel Dokumentation zu allem rund um die Web-Programmierung. Zu JavaScript findet man unter

<https://developer.mozilla.org/de/docs/Web/JavaScript>

Tutorials und Referenzen. Obiger Link leitet auf die deutschen Übersetzungen. Diese sind nicht immer vollständig oder aktuell, daher empfiehlt es sich, im Zweifel die englische Version ebenfalls anzusehen. Generell kann man bei Suchmaschinen einfach einen Suchbegriff gefolgt von „mdn“ angeben, meist ist der erste Treffer dann richtig.

Für TypeScript findet man unter

<https://www.typescriptlang.org/>

ein Handbuch [2] sowie weitere Referenz-Dokumentation.

2. Entwicklungstools



Wenn Sie WE1 bei mir (Jens von Pilgrim) gehört haben, kennen Sie die folgenden Tools und Ausführungen bereits – dann können Sie im Grunde diesen Abschnitt überspringen. Eventuell können Sie aber prüfen, ob Sie die aktuelle Version von Node.js (und NPM) installiert haben und gegebenenfalls updaten.

2.1. Git

Git ist eine **Versionsverwaltung**. Eine Versionsverwaltung speichert dazu verschiedene **Versionen** von Dateien in einem sogenannten **Repository**. Um mit einem Git-Repository zu arbeiten, muss man es von der Quelle, meist Origin genannt und in unserem Fall auf dem GitLab der BHT gespeichert, **klonen** (engl. **clone**). Dabei wird das gesamte Repository auf den lokalen Rechner kopiert, deswegen spricht man bei Git auch von einem **verteilten** Versionssystem.

Wenn man Änderungen an einer oder mehreren Dateien vorgenommen hat, müssen diese der Versionsverwaltung bekannt gegeben werden. Dies wird bei Git über einen sogenannten **Commit** erreicht. Jeder Commit führt zu einer neuen Version, weswegen Version und Commit bei Git auch synonym behandelt werden. Mit einem Commit werden die Änderungen zunächst allerdings nur in die lokale Kopie des Repositories eingespielt. Damit die Änderungen auch im Origin (also bei uns GitLab) sichtbar werden, müssen sie über einen **Push** vom lokalen Repository an das entfernte Repository übertragen werden.

GitLab ist eine Webanwendung, die auf Git aufbaut. Ein Projekt bei GitLab ist einfach ein Git-Repository, das auf einen lokalen Rechner geklont werden kann. Man kann das Git-Repository eines Benutzers auf GitLab auch direkt auf GitLab kopieren – dies nennt man auf Englisch „**forken**“ und die Kopie einen **Fork**. Sie müssen Projekte auf GitLab forken, wenn Sie Änderungen am Projekt durchführen wollen, da Sie keine Schreibberechtigung für die Vorlageprojekte haben. Sie arbeiten dann i. d. R. auf dem Fork, der unter Ihrer Benutzerkennung liegt.

Git ist unter Linux und macOS in der Regel bereits installiert. Unter Windows müssen Sie es eventuell selber installieren. Sie finden die aktuelle Version unter

<https://git-scm.com/>

Gehen Sie auf die Kommandozeile und prüfen Sie dort, ob Git bei Ihnen funktioniert. Sie sollten etwas ähnliches wie im Folgenden sehen:

```
~$ git --version
git version 2.37.0 (Apple Git-136)
~$
```

Grundlagen von Git werden vorausgesetzt. Das Buch „Pro Git“ [3] ist kostenlos und in vielen Sprachen verfügbar. Auf den ersten 60 Seiten lernen Sie alles, was Sie für WE2 brauchen.

2.2. Node.js

Wir werden jetzt Node.js installieren. Laden Sie die Version **26** unter

Node.js 26

<https://nodejs.org/>

für Ihr Betriebssystem herunter, falls Sie bei sich noch kein Node.js installiert haben, und installieren Sie es. Falls Sie eine ältere Version installiert haben sollten, updaten Sie bitte.



Um mit verschiedenen Node-Versionen zu arbeiten und um einfach neue Versionen zu installieren, wird das Tool Node Version Manager (NVM) empfohlen. Wie Sie dieses Tool installieren und verwenden, ist auf der Projektseite des Tools unter <https://github.com/nvm-sh/nvm> beschrieben.

Gehen Sie jetzt auf die Kommandozeile (Terminal bei macOS/Linux oder PowerShell unter Windows). Sie sollten dort nun die Node.js-REPL mit

```
~$ node
```

starten können und in etwa Folgendes sehen:

```
Welcome to Node.js v26.8.1.
Type ".help" for more information.
>
```

Dies ist die sogenannte Read-Eval-Print Loop, in der Sie interaktiv JavaScript ausführen können (genauso wie in der JavaScript-Console beim Browser).

Um die REPL zu beenden, drücken Sie   oder geben Sie `.exit` (mit führendem Punkt!) ein.

Daneben wurde noch ein anderes Tool installiert: NPM. Prüfen Sie einmal, ob auch bei Ihnen alles richtig funktioniert:

```
~$ npm --version
11.19.0
~$
```

Node.js und NPM werden in WE2 vorausgesetzt und nicht weiter erklärt. Die Grundlagen über Node.js und NPM werden von Flanagan [6: Kap. 16.1] erklärt.

2.3. Visual Studio Code

Zur Entwicklung werden wir Visual Studio Code, kurz VS Code, verwenden. Gehen Sie zu

<https://code.visualstudio.com/>

und installieren Sie die aktuelle Version auf Ihrem Rechner. VS Code gibt es für Windows, Linux und macOS. Genauere Hinweise sind unter <https://code.visualstudio.com/docs/setup/setup-overview> zu finden.

Für die Programmierung von JavaScript und TypeScript ist an sich alles vorbereitet. Für einige spezielle Aufgaben werden wir später Extensions installieren. Diese finden Sie in der Seitenleiste unter dem Symbol .

Normalerweise werden Sie dort den Explorer  aktivieren, um Ihre Dateien zu sehen. Um Code nach Git bzw. GitLab einzuspielen, können Sie entweder die Kommandozeile verwenden oder die Source-Control-Dialoge unter . Den Debugger finden Sie (später) unter .

Die vermutlich wichtigste Funktion von VS Code ist die sogenannte „**Command Palette**“. Diese finden Sie im Menü unter  **View**  **Command Palette...** oder über  **Cmd**  **Shift**  (macOS) oder  **Ctrl**  **Shift**  (Windows/Linux).

Eine andere Ansicht, die Sie unbedingt brauchen, ist das „Terminal“. Damit können Sie innerhalb von VS Code auf die Kommandozeile (Bash o.ä. unter macOS/Linux bzw. PowerShell unter Windows) zugreifen. Die Terminal-View können Sie unter dem Menü **Terminal** > **New Terminal** öffnen. Wenn Sie später ein Projekt bearbeiten (also einen Ordner geöffnet haben – bei VS Code sind Projekte einfach Ordner), wird direkt das richtige Verzeichnis gewählt.

2.4. Installieren der Jest-Extension

So, jetzt können wir schon TypeScript programmieren. Da wir aber testgetrieben entwickeln wollen, brauchen wir eine Möglichkeit, Tests bequem zu schreiben und auszuführen. Hierfür verwenden wir das Framework *Jest*².

Jest ist für JavaScript, was JUnit für Java ist. Damit wir unter VS Code komfortabel arbeiten können, installieren wir gleich die passende Extension. Gehen Sie zur Extensions-Ansicht  und geben Sie im Suchfeld „Jest“ ein.

Wählen Sie die richtige Extension aus (Jest, Identifier `orta.vscode-jest`), vgl. Abbildung 1, und klicken Sie auf „Install“.

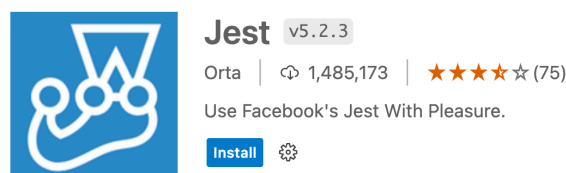


Abbildung 1: Extension Jest, Versionsnummer möglicherweise abweichend

Damit können wir später Tests direkt vom Code aus starten und auch schnell die Ergebnisse sehen – das ist extrem komfortabel!

2.4.1. Konfiguration der Jest-Extension

Wenn man nichts weiter einstellt, beobachtet die Jest-Extension sowohl die eigentlichen Quelltexte (`*.js`, `*.ts`) als auch die Tests (`*.test.js`, `*.test.ts`). Das ist bei kleinen Projekten und schnellen Rechnern sehr komfortabel. Bei etwas größeren Projekten (unser Projekt wird etwas größer) oder langsameren Rechnern ist das allerdings störend, da die Tests zu lange für die Ausführung benötigen und das System andauernd am Laufen ist (und es dann zu Komplikationen kommen kann).

In den Vorgaben ist daher eine Datei `.vscode > settings.json` enthalten, die projektspezifische Konfigurationen enthält. Sie können diese Datei auch gerne selber bearbeiten oder die Einstellungen in Ihre Benutzereinstellungen übernehmen.

Hier ist eine Konfiguration, die Tests nur ausführt, wenn eine Testdatei gespeichert wurde:

```
"jest.runMode": {  
  "type": "on-save",  
  "testFileOnly": true,  
  "deferred": true  
}
```

Wenn Ihr Rechner sehr langsam ist, können Sie das automatische Starten der Tests auch ganz ausschalten:

```
"jest.runMode": "on-demand"
```

Sie können dann die Tests von der Test-View aus (oder über die Anzeigen im Quellcode) manuell starten.

3. Vorbereitung

Forken Sie das Projekt

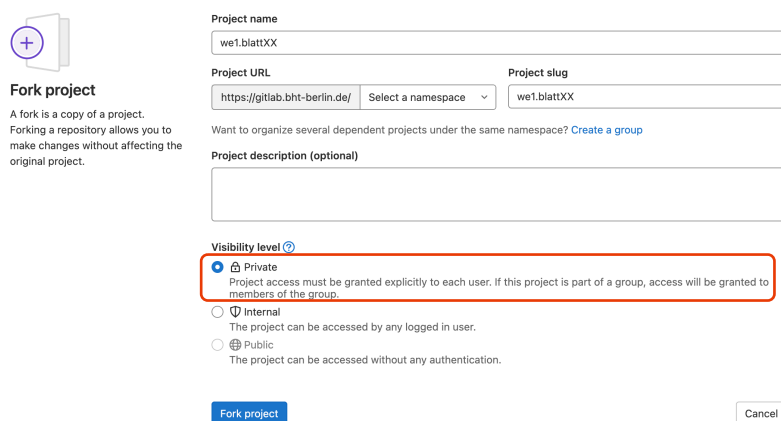
<https://gitlab.bht-berlin.de/we2-26ws/uebung/we2.blatt00>

auf GitLab, stellen Sie die Sichtbarkeit auf ‚private‘ und erstellen Sie eine lokale Arbeitskopie mit `git clone`.

Führen Sie weiterhin die üblichen Schritte aus, v.a. `npm install`.

3.1. Forken

Sie forken das Projekt mit dem Button  Fork 0 auf der GitLab-Seite. Beim Forken unbedingt die **Sichtbarkeit (Visibility) des Projekts auf „Private“ stellen**, wie in Abbildung 2 dargestellt.



Fork project
A fork is a copy of a project.
Forking a repository allows you to make changes without affecting the original project.

Project name
we1.blattXX

Project URL
https://gitlab.bht-berlin.de/

Select a namespace

Project slug
we1.blattXX

Want to organize several dependent projects under the same namespace? [Create a group](#)

Project description (optional)

Visibility level 

☒ Private
Project access must be granted explicitly to each user. If this project is part of a group, access will be granted to members of the group.

☐ Internal
The project can be accessed by any logged in user.

☐ Public
The project can be accessed without any authentication.

[Fork project](#) [Cancel](#)

Abbildung 2: Forken des Projekts und Einstellung der Sichtbarkeit

Falls Sie die Sichtbarkeit beim Forken falsch eingestellt haben, können Sie sie noch in den Einstellungen  **Settings**  **General** korrigieren, vgl. dazu Abbildung 3.

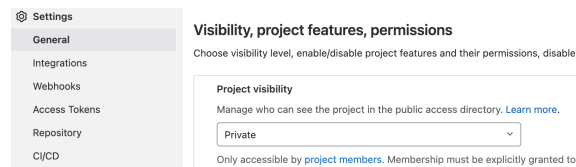


Abbildung 3: Einstellen der Projektsichtbarkeit in Gitlab

3.2. Klonen

Jetzt müssen Sie das Projekt noch auf Ihren Entwicklungsrechner kopieren. Dazu gehen Sie auf die Kommandozeile (Terminal o.ä.) und wechseln dort in das Verzeichnis, in dem das Projekt liegen soll. Das ist nicht das Projektverzeichnis selbst – denn dieses wird von Git automatisch erstellt.



Es wird empfohlen, ein eigenes Verzeichnis `workspace > we2` (o. Ä.) für die Projekte anzulegen – entweder im Hauptverzeichnis einer Festplatte (unter Windows) oder im Benutzerordner (macOS oder Linux). Achten Sie auf jeden Fall darauf, dass der gesamte Pfad keine Umlaute enthält, da dies manchen Tools Probleme bereitet!

Auf der Gitlab-Seite Ihres Forks können Sie nun die Clone-URI unter  ermitteln. Verwenden Sie immer die URL unter „Clone with HTTPS“. Jetzt können Sie das Projekt mit Git klonen. Im folgenden Listing ist dies beispielhaft dargestellt.

```
~$ mkdir we2
~$ cd we2
~/we2$ git clone https://gitlab.bht-berlin.de/sXXXXXX/
@\\projektBlatt@.git
Cloning into 'we2.blatt00'...
...
Resolving deltas: 100% (1/1), done.
~/we2$ ls
we2.blatt00
~/we2$
```

3.3. VS Code starten und Module installieren

Jetzt können Sie das Projekt in VS Code öffnen. Sie können dies direkt über die Kommandozeile machen:

```
~/we2$ code we2.blatt00
```

oder Sie öffnen das Projekt über VS Code über  .

Laden Sie nun die NPM-Pakete. Geben Sie dazu einfach im Terminal (am besten in der Terminal-View von VS Code) im Projektverzeichnis `npm install` ein:

```
~/we2$ cd we2.blatt00
~/we2/we2.blatt00$ npm install

added 335 packages, and audited 336s packages in 2s

found 0 vulnerabilities
~/we2/we2.blatt00$
```

4. Zeilen zählen

Als Erstes müssen Sie eine Funktion `countLines` schreiben. Unter `test › LineCounter.test.ts` finden Sie bereits einen ganz einfachen Test.

Der Test wird nicht funktionieren, da die Quelldatei und die Funktion noch nicht vorhanden sind. Jetzt kommt Ihre eigentliche erste Aufgabe:

Schreiben Sie eine Funktion `countLines` in der Datei

`src › LineCounter.ts`

Den Ordner `src` müssen Sie ggf. selbst erstellen. Diese Funktion nimmt einen String entgegen und gibt die Anzahl der Zeilen im String zurück.

Dabei sind folgende Bedingungen zu beachten:

- Der Funktionskopf ist mit Typannotationen für die formalen Parameter und für das Ergebnis zu versehen. Beachten Sie dabei, dass der übergebene String nicht `null` oder `undefined` sein darf. Dies wird in der Regel durch TypeScript sichergestellt, Sie müssen dafür keine weiteren Abfragen schreiben.
- Es dürfen keine regulären Ausdrücke oder Array-Funktionen verwendet werden. Methoden von String, die auch in Array existieren, dürfen natürlich verwendet werden.
- Wir verwenden die TypeScript- bzw. ES6-Syntax für Module, Export und Import.
- Der Code soll ordentlich formatiert sein. Dies ist am einfachsten durch die Auto-Formatierung von VS Code zu erreichen.

Sie können gerne eigene Tests schreiben!



Sie können in der Test-View  alle Tests über  starten. Falls die Test-View nicht angezeigt wird, haben Sie entweder die Jest-Extension nicht installiert oder es gab andere Probleme (evtl. starten Sie dann mal VS Code neu). In jedem Fall können Sie in der Terminal-View in VS Code den Test-Output ansehen – manchmal findet sich dort die Lösung.

Führen Sie den Test ggf. neu aus. Neben dem Test sollte dann bei Erfolg ein grüner Haken  zu sehen sein. Falls der Test fehlschlägt, wird ein rotes Kreuz

⊗ angezeigt und in der Regel wird ein Hinweis angezeigt, wenn Sie mit der Maus über den rot unterringelten Text fahren. Falls der Test nicht ausgeführt werden konnte, weil beispielsweise ein Syntaxfehler gefunden wurde, dann wird kein Hinweistext angezeigt. Dann ist es sinnvoll, die Problem-View anzusehen oder in der Terminal-View den Test-Output. Falls das alles nicht hilft, können Sie die Tests auch im Terminal mittels

```
~/we2/we2.blatt00$ npm test
```

starten. Die Ausgabe ist im Grunde die Gleiche wie im Test-Output, aber unter Umständen besser lesbar.



Wenn Sie die Funktion geschrieben haben, committen Sie Ihre Änderungen.



Wenn Sie die Funktion geschrieben haben, committen Sie Ihre Änderungen. **Dieser Schritt ist unbedingt notwendig.** Der Commit wird im Commit-Log vermerkt. Das Log wird vom Prüfer auf Plausibilität geprüft. Daher müssen Sie nach jeder Teilaufgabe einen Commit durchführen. Es ist kein Problem, wenn Sie später noch etwas ändern – mehr Commits sind immer erlaubt.

Sie können die Änderungen entweder über die Source-Control  von VS Code committen oder über das Terminal (ebenfalls in VS Code):

```
~/we2/we2.blatt00$ git add .
~/we2/we2.blatt00$ git commit -m "countLines, erste Version"
[master ...] ...
1 files changed...
~/we2/we2.blatt00$
```

Sie können nun die neue Version auch nach Gitlab hochladen, also puschen:

```
~/we2/we2.blatt00$ git push
Enumerating objects: .., done.
Counting objects: 100% (../..), done.
...
Total ..
To https://gitlab.bht-berlin.de/...
xxxxx.yyyy master -> master
~/we2/we2.blatt00$
```



Eigentlich wird erwartet, dass Sie ohne irgendwo nachzusehen schnell selbst Code schreiben können, um durch einen String zu iterieren. Vielleicht waren Sie durch die Zeilenumbrüche irritiert oder Sie sind es quasi gewohnt, vor dem eigenen Denken lieber schnell eine Suchanfrage zu stellen. Dann haben Sie evtl. irgendwo gelesen, dass man mit `split` arbeiten kann. Das geht hier nicht, da die Funktion `split` ein Array liefert – und Arrays dürfen nicht verwendet werden. Für manche Fälle ist die `split` bestimmt die richtige Funktion. Wenn die JavaScript-Engine nicht ein wenig zaubern würde, wäre die `split` hier

extrem ineffizient: Stellen Sie sich einen String mit 50.000 Zeichen vor. `split()` würde dann 50.000 Arrays erzeugen, die man am Ende aber im Grunde gar nicht braucht. Allerdings ist es in der Praxis nicht so schlimm, da die Engine intern optimiert, sodass eine Lösung mit `split` sogar extrem schnell sein kann!

5. Zeilen einer Datei zählen

Sie müssen nun **vier** verschiedene Versionen einer Funktion schreiben, die die Zeilen einer Datei zählt. In allen Fällen sollten Sie die in Abschnitt 4 erstellte Funktion `countLines` aufrufen. Die Funktionen sind alle in der gleichen Datei

📄 [src ▶ LineCounter.ts](#)

abzulegen.

Für alle Funktionen müssen Sie selbst geeignete Tests schreiben. In diesem Aufgabenblatt wird eine Code-Coverage von 100% erwartet!

Für jede dieser Funktionen gilt:

- Die Funktion nimmt als ersten Parameter den vollständigen Pfad (relativ oder absolut inkl. Dateinamen) der Datei entgegen.
- Die Funktion wirft keine Exceptions. Falls ein Fehler auftritt, beispielsweise weil die Datei nicht existiert, wird einfach die Zeilenanzahl `-1` zurückgegeben.
- Bei allen Funktionen ist der Funktionskopf mit Typannotationen für die formalen Parameter und für das Ergebnis zu versehen.
- Schreiben Sie für alle Funktionen mindestens jeweils einen positiven und einen negativen Test! Damit Jest die Tests automatisch findet, müssen die Tests unter `tests` liegen und die Dateien die Endung `„.test.ts“` haben.

Für alle Funktionen werden Sie Funktionen aus der Filesystem-API von Node.js benötigen. Die Dokumentation dieser Funktionen finden Sie unter

<https://nodejs.org/docs/latest/api/fs.html>

Außerdem hilft [6: Kap. 16.7] weiter.



- Bitte nicht `require` verwenden, sondern ausschließlich `import`.
- Die Promise-Varianten zum Dateizugriff müssen von `'fs/promises'`, die synchrone und Callback-Variante von `'fs'` importiert werden. Da eine Funktion in beiden Modulen gleich heißt, muss man mit einem Alias (`as`) arbeiten!
- Bitte daran denken: Einfache Datentypen (String, Number etc.) in TypeScript immer klein schreiben, also `string`, `number` etc.
- Nicht mit globalen Variablen arbeiten!

5.1. Synchrone Variante

Zunächst müssen Sie eine Funktion `countLinesInFileSync` schreiben, die die synchrone Version zum Laden einer Datei der Node.js-API verwendet und auch selbst synchron ist. D. h. als Parameter wird der Name der Datei erwartet; das Ergebnis ist die Anzahl der Zeilen.

 Wenn Sie die Funktion geschrieben haben, committen Sie Ihre Änderungen wie oben beschrieben. Achten Sie auf eine sinnvolle Message, beispielsweise „countLinesInFileSync initial version“.

5.2. Variante mit Callback

Nun müssen Sie eine Funktion `countLinesInFileCallback` schreiben, die die asynchrone Version mit Callback zum Laden einer Datei der Node.js-API verwendet und auch selbst einen Callback (als zweiten Parameter) entgegennimmt. D. h. als erster Parameter wird der Name der Datei erwartet, als zweiter Parameter eine Funktion (die einen Parameter mit der Anzahl der Zeilen erwartet). Die Funktion `countLinesInFileCallback` selbst gibt nichts zurück.

Die Tests werden etwas komplizierter, da Sie nun asynchronen Code testen. Verwenden Sie daher das Argument `done` wie unter

<https://jestjs.io/docs/asynchronous#callbacks>

beschrieben.

 Wenn Sie die Funktion geschrieben haben, committen Sie Ihre Änderungen wie oben beschrieben. Achten Sie auf eine sinnvolle Message.

5.3. Variante mit Promise

Schreiben Sie jetzt eine Funktion `countLinesInFilePromise`, die die asynchrone Version mit einem Promise zum Laden einer Datei verwendet. Diese Funktion selbst soll auch ein Promise zurückgeben. Das heißt, die Funktion hat nur einen Parameter (den Namen der Datei) und gibt ein Promise mit der Anzahl der Zeilen zurück.

Verwenden Sie in dieser Funktion und in den Tests die Schlüsselwörter `async` und `await` noch **nicht!**

Für den Test finden Sie Dokumentation unter

<https://jestjs.io/docs/asynchronous#promises>

 Wenn Sie die Funktion geschrieben haben, committen Sie Ihre Änderungen wie oben beschrieben. Achten Sie auf eine sinnvolle Message.

5.4. Variante mit async/await

Schreiben Sie als letztes eine Funktion `countLinesInFileAsync`, die wieder die asynchrone Version mit Promise zum Laden einer Datei verwendet, selbst als asynchron gekennzeichnet ist und daher `await` verwenden kann.

Für den Test finden Sie Dokumentation unter

<https://jestjs.io/docs/asynchronous#asyncawait>



Committen Sie abschließend Ihre Änderungen.

6. Zählen mit Filter

Ergänzen Sie alle Funktionen so, dass ein letzter optionaler Parameter übergeben werden kann, der einen Filter darstellt. Dieser Filter ist eine Funktion, die einen String (eine Zeile) entgegennimmt und einen Boolean zurückgibt. Nur Zeilen, für die der Filter `true` zurückgibt, sollen gezählt werden.



Achten Sie unbedingt darauf, dass der Parameter optional ist! Die Korrektur wird diese Funktionalität ignorieren.



Diese Aufgabe ist im Grunde nur dazu da, mit genAI zu üben. Falls noch nicht geschehen, sollten Sie unter

<https://education.github.com/pack>

einen Account anlegen und sich für das GitHub Student Developer Pack registrieren. Damit erhalten Sie Zugriff auf weitere genAI-Tools und sogenannte ‚Premium Requests‘ – also umfangreiche Anfragen. Für die ersten Aufgaben sollten die kostenlosen Anfragen ausreichen.

Öffnen Sie die GitHub Copilot Chat Extension in VS Code, entweder über **View** > **Command Palette...** und dort „GitHub Copilot Chat: Open Chat“ eingeben, oder über Shortcut **⌘ Cmd** **⇧ Shift** **I** (macOS) oder **^ Ctrl** **⇧ Shift** **I** (Windows/Linux). Dort können Sie nun Anfragen, sogenannte Prompts, eingeben.

Ganz unten (unter der Eingabezeile) können Sie das Modell auswählen. Claude Sonnet 4.6 kann gut mit Code umgehen, noch besser ist Claude Opus 4.6 (was allerdings teurer in der Verwendung ist und ggf. in der kostenlosen Version nicht verfügbar ist).



genAI möglichst im DevContainer betreiben

Bei den ersten Blättern wird es vermutlich noch nicht so schlimm sein, wenn Sie genAI außerhalb des DevContainers verwenden, da die Anfragen eher einfach sein werden und die genAI durch einfache Analyse Lösungen direkt im Code erstellen kann.

Bei komplizierteren Aufgaben verwendet die genAI dann Agenten und jede Menge Tools – von einfachen Tools wie `grep` oder `sed` zur Analyse der Codebasis bis hin zu kleinen, schnell geschriebenen Skripten, häufig in Python. Das Problem ist: Sie sehen oft nicht, was wirklich passiert. Und leider sind die Tools nicht perfekt. So reicht es schon, dass das Tool temporäre Verzeichnisse mit `rm -rf` löschen will und dabei vergisst, dass es in den Home-Folder gewechselt ist ...

Wenn Sie im DevContainer arbeiten, ist das dann nicht ganz so schlimm, da das Tool im Container ausschließlich Zugriff auf das aktuelle Projekt hat (und hier sollten alle Ergebnisse ja sowieso in Git bzw. GitLab liegen).

7. Einreichung der Lösung

Jetzt müssen Sie Ihre Lösung einreichen. Das Verfahren ist für alle Blätter gleich und wird hier sehr ausführlich beschrieben – im Grunde ist es aber ganz einfach.



Hier die Abgabe in kurzen Stichpunkten:

1. `npm run build` muss erfolgreich durchlaufen.
2. `npm test` muss verlangte Code-Coverage anzeigen.
3. `npm run ailog` muss die Datei `ailog.md` erzeugen, `git add ailog.md` und `git commit -am "ailog"` durchführen.
4. `git push` durchführen und Gitlab-Pipeline prüfen.
5. `npm run abgabe` durchführen, um `abgabe_Blatt_00.zip` zu erzeugen.
6. `abgabe_Blatt_00.zip` bis 23:59 bei Moodle hochladen.

Bitte beachten Sie die Hinweise, da ansonsten Ihre Abgabe nicht bewertet werden kann.

1. Achten Sie darauf, dass der Code übersetzt werden kann. D. h.

```
~/we2/we2.blatt00$ npm run build
```

muss ohne Fehler durchlaufen. Ansonsten kann Ihre Abgabe nicht bewertet werden und Sie erhalten 0 Punkte (bzw. ab Blatt 1 die Note 5).

2. Überprüfen Sie die Code-Coverage. Wenn Sie die Tests im Terminal ausführen, sollte am Ende eine Coverage angezeigt werden:

```
~/we2.blatt00$ npm test
...
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files |    100  |     100  |     100  |     100  |
LineCounter.ts |    100  |     100  |     100  |     100  |
...
~/we2.blatt00$
```

Gesamte Statement-Coverage

Bei der geforderten Coverage handelt es sich immer um die Statement-Coverage für alle Dateien („Stmts“, hier rot hervorgehoben).

Es gibt Tests in `tests > LineCounter.formal.test.ts`, die prüfen, dass Sie korrekte Deklarationen angegeben haben. Wenn diese Tests Fehler zeigen, sind diese unbedingt zu korrigieren, da ansonsten die automatische Korrektur auch nichts findet und Sie (später, wenn es zählt) 0 Punkte bekommen würden.

3. Erstellen Sie die Copilot Chat History. Dieses Skript bitte nicht im DevContainer ausführen, da es auf die lokale Chat-History zugreift – und diese ist im DevContainer nicht sichtbar. Wechseln Sie also auf Ihrem lokalen Rechner in das Projektverzeichnis und führen Sie dort folgendes Skript aus:

```
~/we2.blatt00$ npm run ailog
```

Dieser Befehl erzeugt, falls Sie mit Copilot Chat gearbeitet haben, die Datei

📄 `ailog.md`

Diese Datei müssen Sie abgeben. Diese Datei enthält die gesamte Chat-History, also alle Prompts und Antworten, die Sie mit der GitHub Copilot Chat Extension geführt haben.

Überprüfen Sie, dass die Datei tatsächlich vorhanden ist und die komplette Chat-History enthält. Falls in der Datei Einträge fehlen, beenden Sie VS Code und versuchen Sie es dann noch einmal.

Falls dieser Aufruf keine Datei erstellt, kann dies mehrere Ursachen haben:

- Sie haben tatsächlich **nicht mit genAI gearbeitet**: In diesem Fall erstellen Sie bitte die Datei manuell und geben dort an: „Ich habe den Code vollständig selbst geschrieben und nicht mit einer generativen KI gearbeitet.“.
- **Sie arbeiten im Dev Container**: In diesem Fall führen Sie diesen Befehl im Projektverzeichnis auf Ihrem lokalen Rechner aus. Unter Windows kann es sein, dass Sie das Tool direkt aufrufen müssen, also mittels

```
~/../we2.blatt00> node  
"node_modules\copilothistory\bin\cli.js" export -o  
ailog.md --remotePath /workspaces
```

Der Aufruf erfolgt in einer einzigen Zeile; die Umbrüche hier dienen nur der besseren Lesbarkeit.

- **In allen anderen Fällen** gehen Sie bitte wie folgt vor:
 - Öffnen Sie VS Code (im Dev Container oder auf Ihrem Rechner, je nachdem, wo Sie die genAI genutzt haben) und bitten Sie dort die genAI, die Datei zu erstellen: „Please create a file ‚ailog.md‘ with the history of our chat. Include all messages and the date and time of each message.“
 - Sprechen Sie in der nächsten Übung den Dozenten an.



Fügen Sie die Datei 📄 `ailog.md` hinzu und committen Sie sie.



Die Abgabe der Datei 📄 `ailog.md` ist verpflichtend. Sie wird in die Bewertung einbezogen. Falls die Datei fehlt, wird die Aufgabe mit 0 Punkten bewertet.

4. Achten Sie darauf, dass **alle Änderungen committet** wurden. Pushen Sie nach GitLab. Dies ist für die Abgabe nicht unbedingt erforderlich, denn die Abgabe hat mit GitLab nichts zu tun. Allerdings können Sie so sicherstellen, dass Ihr Code auch auf anderen Rechnern läuft und Sie beispielsweise nicht vergessen haben, etwas ins Repository einzuspielen. Das Projekt ist so konfiguriert, dass die Tests auch auf GitLab ausgeführt werden. Sie können

unter **Build** **Pipelines** die Ergebnisse der Tests auf GitLab sehen. Klicken Sie auf das Banner, i. d. R.  Passed oder  Failed, um die Details zu sehen. Wenn Sie unter dem Reiter **Pipeline** (und auch unter dem Reiter **Jobs**) auf das Banner „test“ bzw. „javascript“ klicken, können Sie die Testausgabe sehen, so wie Sie sie auch im Terminal sehen würden. Unter dem Reiter **Test** können Sie die Details der Tests sehen, wobei die fehlgeschlagenen Tests oben angezeigt werden.

Falls die Tests auf GitLab andere Ergebnisse zeigen als die lokale Ausführung, dann haben Sie vermutlich vergessen, etwas zu committen oder zu pushen. Am besten rufen Sie dann lokal `git status` auf, um eventuell vergessene Dateien zu sehen.

5. Wenn alles soweit in Ordnung ist, erzeugen Sie über das Skript „abgabe“ eine Zip-Datei Ihres Projekts:

```
~/we2.blatt00$ npm run abgabe
```

Dieses Skript erzeugt die Datei `abgabe_Blatt_00.zip`, die Sie abgeben müssen.

Verwenden Sie unbedingt dieses Skript zur Erstellung der Zip-Datei. Die Zip-Datei enthält einfach Ihr lokales Git-Repository. Außerdem prüft das Skript, dass Sie wirklich alle Änderungen committet haben. Außerdem ist so sichergestellt, dass die Zip-Datei das richtige Format und auch ansonsten korrekte Inhalte hat.



Datenschutzhinweise

Die zur Abgabe erstellte Zip-Datei enthält, neben Ihrer Chat-History, auch eine automatisch erstellte Chat-History! Dieser Hinweis ist für Sie zur Information, damit Ihnen klar ist, was bei der Abgabe enthalten ist.

6. Gehen Sie zum Lernraum von WE2 in Moodle und laden beim Aufgabenblatt die Dateien `abgabe_Blatt_00.zip` hoch. Dies muss immer am Tag der Abgabe bis 23:59 Uhr geschehen. Später eingereichte Abgaben werden nicht mehr bewertet. Es ist vielleicht sinnvoll, nicht bis zur letzten Sekunde zu warten, falls Moodle mal etwas langsam ist.

8. Abnahme und Bewertung



Dieses Aufgabenblatt fließt nicht in die Bewertung ein. Es dient dazu, mit den Tools warm zu werden, die Arbeitsweise mit Tests zu üben, die Abgabe zu testen und das Gesamtverfahren kennenzulernen. Dieses Aufgabenblatt ist noch kein Teilleistungsnachweis im Sinne der Prüfungsordnung [1: §22 (2)]. Damit können Sie mit diesem Blatt für sich prüfen, ob Sie bereit für WE2 sind.

Es gelten die allgemeinen Modalitäten, wie sie im Blatt „WE2 Übung Modalitäten“ beschrieben wurden.

Abkürzungsverzeichnis

API	Application Programming Interface
NPM	Node Package Manager, https://www.npmjs.com/
NVM	Node Version Manager, https://github.com/nvm-sh/nvm
REPL	Read-Eval-Print Loop
TDD	Test-Driven Development, auf Deutsch testgetriebene Entwicklung
WE1	Web-Engineering 1
WE2	Web-Engineering 2
genAI	generative Artificial Intelligence, dt. generative künstliche Intelligenz

Literatur

- [1] *Rahmenstudien- und Prüfungsordnung (RSPO 2016) der Beuth-Hochschule für Technik Berlin*. Amtliche Mitteilung (37. Jahrgang, Nr. 16), 2016. URL https://www.bht-berlin.de/fileadmin/oe/pressestelle/amtliche_mitteilung/2016/amtliche_mitteilung_16-2016.pdf
- [2] *The TypeScript Handbook*. URL <https://www.typescriptlang.org/docs/handbook>
- [3] CHACON, SCOTT; STRAUB, BEN: *Pro Git*. deutsch, 2nd edition (CC BY-NC-SA 3.0), 2014. URL <https://git-scm.com/book/de/v2>
- [4] CHERNY, BORIS: *Programmieren in TypeScript*. O'Reilly, 2019 — ISBN 978-3-96010-362-2
- [5] ECMA INTERNATIONAL: *ECMAScript 2024 Language Specification*. Standard ECMA-262, 2024. URL https://ecma-international.org/wp-content/uploads/ECMA-262_15th_edition_june_2024.pdf
- [6] FLANAGAN, DAVID: *JavaScript – Das Handbuch für die Praxis*. 7. Auflage. O'Reilly, 2021 — ISBN 9781098129125